PhysSplat: A Physically Grounded 3D Neural World Model for Interactive Environments

Technical Design Document

Mohamad Salman

August 2026

Abstract

PhysSplat converts a photograph of everyday rigid objects on a flat surface (blocks, pencils, erasers, and similar simple items) into an interactive, physically plausible 3D simulation that runs in a web browser. The pipeline lifts the image into a 3D Gaussian Splat representation, decomposes the splat cloud into discrete rigid bodies without supervision, and simulates those bodies with a learned graph-network dynamics model trained entirely on synthetic data. The user can orbit the scene, poke and shove objects with the mouse, and watch them topple, slide, and roll, with every frame rendered from the same Gaussians that were reconstructed from the photo. This document specifies the full architecture, the mathematics of the learned simulator, the synthetic data pipeline, and a step-by-step build plan starting from an empty repository on an Apple M4 Pro with a compute budget under $30.

Contents

1 What the Project Is and What It Demonstrates

1.1 The one-sentence version

Take a photo of a few simple objects on a table (blocks, pencils, erasers), get back a browser window where those exact objects exist in 3D and obey physics you can interact with, where the physics is a neural network rather than a hand-coded solver. The scope is deliberately bounded to *convex-ish rigid objects on a flat surface*: things that stack, slide, tip, and roll. Concave or articulated objects (mugs, scissors) are out of scope.

1.2 Why this is interesting

Two mature technologies exist on either side of this project, and neither crosses the gap alone:

- **Generative 3D reconstruction** (3D Gaussian Splatting, feed-forward image-to-3D models) produces photorealistic 3D scenes from images, but the output is a hollow visual shell. Nothing has mass, nothing collides, gravity does not exist.
- **Physics engines** (MuJoCo, PyBullet, Rapier) simulate rigid bodies robustly, but they require clean meshes, convex decompositions, and hand-tuned parameters. They cannot ingest a photograph.

PhysSplat bridges the two with a *learned* dynamics model: a graph neural network trained on synthetic trajectories that predicts how bodies move under gravity, contact, friction, and user-applied impulses. No meshing, convex decomposition, or manual parameter tuning is needed at inference time.

1.3 The world model formulation

The system learns an implicit transition function

$$s_{t+1} = f_\theta(s_t, a_t), \tag{1}$$

where the state s_t is a set of particles with positions, velocities, and body assignments; the action a_t is an external impulse injected by the user at a 3D location; and f_θ is a graph neural network that models contact through message passing rather than analytic collision resolution. This is the defining property of a world model: the dynamics are learned from data, not programmed.

1.4 What it demonstrates to a technical reviewer

- **Spatial intelligence:** lifting unstructured 2D pixels into a structured, object-decomposed 3D representation.
- **Geometric deep learning:** a graph network simulator with relational inductive biases, rigid-body structure, noise-robust multi-step rollout, and correct input/output normalization.
- **ML systems engineering:** synthetic data generation at scale, training on Apple Silicon via Metal Performance Shaders, ONNX export, and client-side neural network inference in the browser at interactive rates.
- **Full-stack execution:** a deployed Next.js application with a real-time WebGL Gaussian renderer.

2 System Architecture

Stage	Runs	Frequency
A. Image → 3D Gaussians	Server / local Mac	once per scene
B. Decomposition into rigid bodies	Server / local Mac	once per scene
C. Physics particle resampling & canonicalization	Server / local Mac	once per scene
D. Learned dynamics rollout (GNN)	**Browser (ONNX Runtime Web)**	60 Hz
E. Gaussian rendering & interaction	Browser (WebGL/WebGPU)	60 Hz

Data flow: a photo is uploaded; Stage A produces a splat file; Stage B labels every Gaussian with a body id and extracts the ground plane; Stage C samples physics particles per body and normalizes the scene (gravity-aligned, metric-scaled); the browser receives one packet containing the labeled Gaussians, the physics particles, and body metadata, and from then on everything runs client-side: the GNN steps the particles, rigid transforms are recovered per body, the Gaussians are moved, sorted, and splatted.

Two design decisions shape everything downstream:

Two coupled representations. The physics does not run on the raw Gaussians. A reconstruction produces tens of thousands of nonuniformly dense Gaussians, which is too slow for a radius-graph GNN and statistically unlike any training data. Instead, each body gets a small set of uniformly resampled *physics particles* (Section 3.3) that the GNN simulates, and the full Gaussian set is carried along rigidly: each frame, body b's transform $(\mathbf{R}_b, \mathbf{t}_b)$ is applied to its Gaussians as $\boldsymbol{\mu}' = \mathbf{R}_b\boldsymbol{\mu} + \mathbf{t}_b$ and $\boldsymbol{\Sigma}' = \mathbf{R}_b\boldsymbol{\Sigma}\mathbf{R}_b^\top$.

Client-side inference. Round-tripping physics steps through a server adds 50 to 300 ms of latency per step and destroys interactivity. The dynamics network is small (about 5M parameters over about 1,000 particles), so it runs in the browser via ONNX Runtime Web at a few milliseconds per step. The server's only job is the one-time reconstruction, which is latency-tolerant.

Training happens entirely offline (Section 7) on synthetic PyBullet scenes that mimic Stage C's output format, which is what makes the sim-to-recon transfer work.

3 The Pipeline, Stage by Stage

3.1 Stage A: Image to 3D Gaussians

Primary path (single image). A feed-forward image-to-3D model runs once per scene:

- **LGM** [4]: generates multi-view images with a diffusion model, then regresses Gaussians directly. Native Gaussian output. Heavier, and the multi-view diffusion step is the flakiest part on MPS; if MPS misbehaves, run this one stage on a rented GPU or free Colab, since it is offline and one-shot.
- **TripoSR** [5]: faster and more robust, but outputs a triplane field, so we extract a mesh (marching cubes), sample its surface, and instantiate one small isotropic Gaussian per sample with the surface color. Slightly "point-cloudy" visuals but serviceable.

Both are trained on single centered objects. A compact tabletop arrangement of a few small objects against a plain background reads as one object, which these models handle acceptably; the occluded

back is hallucinated, but simple convex shapes make this rarely noticeable. Background removal (`rembg`) is applied before reconstruction. One caution: very thin structures (a lone pencil) are the weakest case for feed-forward reconstructors and may come back lumpy; the physics tolerates this (the convex hull of a lumpy pencil is still pencil-shaped), but for hero shots prefer chunkier objects or the multi-view path.

Fallback path (multi-view). 30 to 60 phone photos orbiting the scene, processed by a standard 3DGS pipeline [3], yield a far higher-fidelity splat scene including a real ground plane. The downstream pipeline is identical.

Output: a `.ply` of N Gaussians, each with position $\boldsymbol{\mu}_i \in \mathbb{R}^3$, covariance (stored as scale and quaternion), opacity, and spherical-harmonic color coefficients.

3.2 Stage B: Unsupervised decomposition into bodies

1. **Gravity alignment and scaling.** Align the dominant plane normal to $+z$ and rescale so object sizes are plausible in metric units (the object class is known, so metric scale is assumed rather than estimated).

2. **Ground extraction.** RANSAC plane fit on the lowest 30% of points (by z). Inliers are labeled *static ground*. If the single-image path reconstructs no ground, a synthetic plane is added at the scene's minimum z.

3. **Clustering.** HDBSCAN over $[\,\boldsymbol{\mu}_i/\sigma_{xyz},\ \lambda\,\mathbf{c}_i\,]$, normalized position concatenated with weighted RGB from the DC spherical-harmonic term. Position separates spatially distinct objects; color separates touching objects of different colors. Clusters below a minimum size are merged or discarded as floaters.

4. **Per-body statistics.** Centroid, principal axes (PCA), an oriented bounding box (used for mouse-ray picking), and mass plus inertia estimated from the cluster's convex hull with an assumed density.

Output: a body id per Gaussian, a ground label, and per-body metadata.

3.3 Stage C: Physics particle resampling and canonicalization

This stage makes real reconstructions statistically indistinguishable from training data, which is the entire trick behind sim-to-real transfer here.

For each body, sample surface points by farthest-point sampling at a *fixed target spacing* of $\sim$4 mm (capped at $P_{\max} \approx 400$ per body), rather than a fixed count. Fixed spacing keeps neighborhood statistics consistent across object sizes: a pencil and a large block have the same local particle density even though their counts differ, so the contact radius stays a single global constant. The sampler must be *byte-for-byte the same procedure* used when generating training data. Each particle stores its offset from the body centroid in the body frame, which is what lets us recover the body's rigid transform. Ground is handled analytically as a node feature rather than with ground particles.

Output: particle positions $\mathbf{x}_i$, body ids b_i, body-frame offsets $\mathbf{r}_i$, per-body mass and inertia, and zero initial velocities.

4 The Learned Dynamics Engine

The core of the project: an Encode–Process–Decode graph network in the GNS family [1], with a rigid-body output head. Classic GNS predicts an independent acceleration per particle, which works for sand and water but fails for rigid bodies: accumulated per-particle error violates rigidity within tens of steps and objects visibly melt. PhysSplat therefore keeps the particle-level contact graph but constrains the output to rigid motion, following the per-body approach of FIGNet [2].

4.1 Graph construction (every step)

Nodes are the physics particles. Edges connect pairs within a contact radius:

$$\mathcal{E}_t = \{(i,j) : \|\mathbf{x}_i - \mathbf{x}_j\| < r_{\text{contact}},\ i \neq j\}, \tag{2}$$

with r_{contact} about 1.5× the particle spacing. Same-body edges propagate information across a body; cross-body edges carry contact, and an edge feature flags which kind each edge is. Neighbor search uses a uniform grid hash, $O(N)$ per step.

4.2 Input features and normalization

Per-node input:

$$\mathbf{h}_i^{(0)} = \left[\ \mathbf{v}_i^{t-C+1}, \ldots, \mathbf{v}_i^{t} \ \middle\|\ d_i^{\text{ground}} \ \middle\|\ m_{b_i} \ \middle\|\ \text{diag}(\mathbf{I}_{b_i}) \ \middle\|\ \mathbf{a}_i^{\text{ext}}\ \right], \tag{3}$$

a history of $C = 5$ finite-difference velocities, a clipped distance to the ground plane $d_i^{\text{ground}} = \min(z_i, r_{\text{contact}})$ (how the network senses the floor without floor particles), the body mass, the diagonal of the body's inertia tensor in its principal frame (a pencil and a cube respond very differently to the same push; the network could infer this from geometry, but providing it is cheap and helps), and the external impulse feature $\mathbf{a}_i^{\text{ext}}$ (zero except during user interaction, Section 5.3). Absolute positions are never input, giving translation invariance for free.

Per-edge input for (i,j): relative displacement $\mathbf{x}_j - \mathbf{x}_i$, its norm, and the same-body flag.

Every input and target dimension is standardized by dataset statistics. In GNS-class models this is not a nicety; it is the difference between converging and not, because raw accelerations are dominated by gravity while contact spikes are orders of magnitude rarer.

4.3 Encoder, processor, decoder

Encoder: two-layer MLPs lift node and edge features to $d = 128$-dimensional latents.

Processor: $L = 10$ message-passing blocks with residual connections:

$$\mathbf{e}_{ij}^{(\ell+1)} = \mathbf{e}_{ij}^{(\ell)} + \phi_e\big(\mathbf{e}_{ij}^{(\ell)}, \mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}\big), \tag{4}$$

$$\mathbf{h}_i^{(\ell+1)} = \mathbf{h}_i^{(\ell)} + \phi_v\Big(\mathbf{h}_i^{(\ell)}, \textstyle\sum_{j \in \mathcal{N}(i)} \mathbf{e}_{ij}^{(\ell+1)}\Big), \tag{5}$$

where ϕ_e, ϕ_v are two-layer MLPs with LayerNorm. Ten hops let a contact on one corner of a body influence the opposite corner within a single step. Implementation is plain PyTorch (edge MLPs on gathered node pairs, aggregation via `index_add_`); PyTorch Geometric is deliberately avoided because its scatter kernels have poor MPS coverage and complicate ONNX export.

Decoder (per-body rigid head): mean-pool final node latents within each body b and decode

$$(\Delta \mathbf{v}_b,\ \Delta \boldsymbol{\omega}_b) = \psi\Big(\tfrac{1}{|b|}\textstyle\sum_{i\in b} \mathbf{h}_i^{(L)}\Big) \in \mathbb{R}^6, \tag{6}$$

a linear and angular acceleration at the body's center of mass. Because every particle of a body moves under one SE(3) update, rigidity is exact by construction and rollouts cannot melt.

4.4 Integration

Semi-implicit Euler per body, with gravity added analytically so the network only learns the *residual* dynamics (contact and friction), an easier function:

$$\mathbf{v}_b^{t+1} = \mathbf{v}_b^t + \mathbf{g}\,\Delta t + \Delta \mathbf{v}_b, \qquad \boldsymbol{\omega}_b^{t+1} = \boldsymbol{\omega}_b^t + \Delta \boldsymbol{\omega}_b, \tag{7}$$

$$\mathbf{t}_b^{t+1} = \mathbf{t}_b^t + \mathbf{v}_b^{t+1}\Delta t, \qquad \mathbf{R}_b^{t+1} = \exp\big([\boldsymbol{\omega}_b^{t+1}]_\times \Delta t\big)\,\mathbf{R}_b^t, \tag{8}$$

and particle positions follow: $\mathbf{x}_i^{t+1} = \mathbf{R}_b^{t+1}\mathbf{r}_i + \mathbf{t}_b^{t+1}$.

4.5 The shape-matching fallback

If the per-body head underfits (pooling discards where on the body contact happened), a fallback decoder predicts per-particle accelerations as in classic GNS, integrates particles freely, then snaps each body back to rigidity via the Kabsch projection: with predicted positions $\tilde{\mathbf{x}}_i$ and offsets $\mathbf{r}_i$,

$$\mathbf{R}_b^* = \arg\max_{\mathbf{R}\in SO(3)} \operatorname{tr}\big(\mathbf{R}^\top \mathbf{A}_b\big), \qquad \mathbf{A}_b = \sum_{i\in b} (\tilde{\mathbf{x}}_i - \bar{\mathbf{x}}_b)\,\mathbf{r}_i^\top, \tag{9}$$

solved by SVD of $\mathbf{A}_b$, with $\mathbf{t}_b^* = \bar{\mathbf{x}}_b$. Both heads share the same encoder and processor, so switching is a one-line config change.

4.6 Loss and rollout stability

Training minimizes single-step MSE on normalized accelerations. Single-step training alone drifts over long rollouts because the model never sees its own errors; two standard remedies are used:

- **Training-noise injection:** corrupt input positions and velocities with random-walk Gaussian noise ($\sigma \approx 1$ to 3 mm over the history window) while computing targets against the clean next state. The model learns to *correct* perturbations, exactly the skill rollout requires. This is the highest-leverage trick in the GNS literature.
- **Short fine-tuning rollouts:** after single-step convergence, fine-tune on 4 to 8-step unrolled losses at a small learning rate.

5 Browser Runtime, Rendering, and Interaction

5.1 Client-side inference

The trained model is exported to ONNX with symbolic shapes for dynamic node and edge counts. Scatter aggregation exports to ONNX `ScatterND`/`ScatterElements`; the SVD in the fallback head does not export, so if that head ships, the Kabsch projection is implemented once in TypeScript (30 lines on 3×3 matrices). Graph construction (grid-hash neighbor search) also lives in TypeScript and rebuilds the edge list each step. Runtime: ONNX Runtime Web with the WebGPU execution provider where available, WASM-SIMD otherwise. At ∼1,000 nodes and 5M parameters, a step costs single-digit milliseconds; physics runs at 60 Hz, or 30 Hz with interpolation.

5.2 Rendering moving Gaussians

A Three.js / React Three Fiber scene using an existing 3DGS splatting library (GaussianSplats3D or Spark). Each body's Gaussians form their own splat batch with a per-batch model matrix, so the CPU uploads 6 floats per body per frame rather than rewriting buffers; covariances rotate by conjugation with $\mathbf{R}_b$ (in practice, rotate the stored quaternion). View-dependent depth sorting is retriggered when bodies move, which the mature libraries handle.

5.3 Interaction

Mouse down casts a ray tested against per-body OBBs from Stage B. A hit turns the drag vector into an impulse: direction in the camera plane, magnitude from drag length, capped to the training distribution's maximum. The impulse enters the next $K \approx 3$ physics steps as $\mathbf{a}_i^{\text{ext}}$ on the hit body's particles, weighted by Gaussian falloff from the hit point, so poking a corner produces spin. Sliders for gravity scale and impulse strength, and a reset button (free, since the initial state is just the Stage C packet).

6 Synthetic Training Data

All training data is generated locally with PyBullet on CPU, deterministic seeds. Nothing is downloaded or labeled.

6.1 Scene distribution

Each trajectory:

- 2 to 6 objects drawn from a randomized *primitive-shape family*: boxes (erasers, blocks), cylinders and capsules (pencils, markers, batteries), and random convex polyhedra. Longest dimension 2 to 16 cm, aspect ratios up to 15:1 to cover pencil-like shapes; friction $\mu \in [0.3, 0.9]$; restitution $\in [0.0, 0.4]$; density fixed. Convexity is the scope boundary: any real object is simulated as its convex hull, which is accurate for the target class.
- Initial conditions mixed across three regimes: (a) stable-ish stacks (settled 100 steps before recording), (b) objects lying scattered flat on the surface, the common case for pencils and erasers, and (c) random drops from low height. Cylinders must appear often enough at rest and mid-roll that the model learns both rolling and coming to rest, the hardest low-energy behavior in this domain.
- At 1 to 3 random times, an impulse of random magnitude and direction applied at a random surface point of a random object, recorded as the action channel. This teaches the response to user pokes.
- 300 steps recorded at $\Delta t = 1/60\,\text{s}$ (simulated at 240 Hz internally, subsampled), storing per-body poses and twists.

Target: 5,000 trajectories $\rightarrow \sim$1.5M transitions, an hour or two on the M4 Pro's CPU cores with multiprocessing.

6.2 Particle-ization

For every trajectory, particles are sampled once on each object's surface mesh with the exact Stage C sampler (farthest-point at fixed spacing), stored as body-frame offsets; world positions at every step derive from the recorded poses. Storing poses plus offsets instead of raw point clouds keeps the dataset in the tens of MB and guarantees rigid-consistent ground truth.

6.3 Domain randomization for the reconstruction gap

Reconstructed objects are not perfect primitives. At training time: perturb particle positions with 0.5 to 2 mm surface noise, randomly delete 5 to 15% of particles, and jitter per-body counts, so a lumpy reconstructed cluster still looks in-distribution.

7 Training and Evaluation

Latent width / MP layers	128 / 10
Parameters	$\approx$ 5M
Optimizer	AdamW, lr $10^{-4} \to 10^{-6}$ (exponential decay)
Batch	2 to 4 scenes' worth of graphs per step
Steps	300k to 500k single-step, then 20k rollout fine-tune
Precision	float32 (MPS has no float64)
Hardware	M4 Pro MPS; optionally one A100 day ($< \$30$) for the final run
Wall clock	1 to 3 days on MPS

Practical notes: run with `PYTORCH_ENABLE_MPS_FALLBACK=1`; build graphs in dataloader workers, not the training loop; log validation *rollout* error every few thousand steps, since single-step loss is a poor proxy for what matters.

7.1 Evaluation metrics

- **Rollout error:** mean per-body translation and rotation error vs. ground truth at 50, 150, 300 steps.
- **Penetration depth:** mean and max interpenetration between bodies and with the ground; the learned model has no hard constraints, so this quantifies how well contact was learned.
- **Stability:** fraction of initially stable scenes that remain at rest after 300 unperturbed steps. A model that knocks over stable stacks by itself is not ready.
- **Energy sanity:** total energy must not grow over a passive rollout.
- **Qualitative:** side-by-side videos, learned vs. PyBullet, same seed and same pokes.

8 Step-by-Step Build Plan from Zero

Milestones are ordered so that every one ends with something visible and the riskiest work (the learned simulator) happens first; the reconstruction front end, which is mostly running pretrained checkpoints, comes after. Time estimates assume part-time work.

8.1 Milestone 0: environment and skeleton (half a day)

```
uv init physsplat && cd physsplat
uv add torch numpy scipy pybullet h5py rerun-sdk hdbscan trimesh onnx
   onnxruntime
python -c "import torch; print(torch.backends.mps.is_available())"
```

```
physsplat/
  datagen/     # PyBullet scene generation, particle sampling
  model/       # GNN, integrators, losses (plain PyTorch)
  train/       # loops, config, checkpoints
  recon/       # image -> splats -> bodies -> particles
  export/      # ONNX export + parity tests
  web/         # Next.js app (Milestone 5)
  scripts/     # entry points
```

Install the Rerun viewer early: logging 3D particle states to Rerun is the fastest way to inspect everything this project produces without writing viewer code.

8.2 Milestone 1: synthetic data you can watch (week 1)

1. `datagen/scenes.py`: random scene generator over the primitive-shape family (start with boxes only, add cylinders and convex hulls once the loop works), settle, record poses at 60 Hz, apply logged impulses. Verify visually in PyBullet's GUI before scaling.

2. `datagen/particles.py`: farthest-point fixed-spacing surface sampler over an arbitrary mesh (trimesh handles primitives and hulls uniformly), body-frame offsets. Reused verbatim in Stage C; keep it shared.

3. HDF5 writer (poses, offsets, actions, material parameters) and a loader that reconstitutes world-space particles.

4. Generate 50 trajectories, log a few to Rerun, and check: no interpenetration in ground truth, impulses visibly kick objects, scenes settle.

5. Scale to 5,000 trajectories across the full shape family. Compute and store normalization statistics.

Deliverable: a Rerun recording of ground-truth physics from your own generator.

8.3 Milestone 2: the learned simulator (weeks 2–3, the crux)

1. Graph builder (grid-hash radius search), unit-tested against brute-force $O(N^2)$.

2. Encode–Process–Decode with both heads (per-body, and per-particle plus Kabsch). Plain PyTorch.

3. Overfit a single trajectory to near-zero loss. If this fails, the bug is in features, normalization, or indexing, and no amount of data will save it.

4. Train on the full set with noise injection. Targets in order: a dropped box lands and stops; a two-box stack stays up; a poked stack topples plausibly; a flicked pencil rolls and comes to rest.

5. Rollout evaluation harness (metrics of Section 7) and side-by-side videos.

6. Rollout fine-tuning pass.

Deliverable: video, learned vs. PyBullet, same poke, visually matching for 5 seconds. Once this exists the project cannot fail completely; everything after is integration.

8.4 Milestone 3: interactive local demo, synthetic scenes (week 4)

A minimal Three.js page (objects as plain meshes, no splats yet) connected over WebSocket to a Python process running the model on MPS. Click to poke. This validates the interaction loop, the impulse feature, and reset, with none of the web-inference complexity. **Deliverable:** you can knock over a synthetic stack with your mouse, locally.

8.5 Milestone 4: the reconstruction front end (week 5)

1. Photograph 3 to 5 distinctly colored objects (blocks and erasers first; add pencils once boxes work), plain background, good light. Run `rembg`, then TripoSR (easier), LGM if quality disappoints. Export Gaussians or points as `.ply`.

2. Stage B: gravity align, RANSAC ground, HDBSCAN with position+color features, per-body OBB, mass, inertia. Tune λ and `min_cluster_size` on your test scene. Log colored-by-body clouds to Rerun.

3. Stage C with the shared sampler from Milestone 1. Verify in Rerun that reconstructed-scene particles look like training particles.

4. Feed the result to the Milestone 3 demo. Expect one iteration round (scale and gravity mismatches are the usual suspects; domain randomization is the usual fix).

Deliverable: photo in, interactive (mesh-rendered) physics out, locally.

8.6 Milestone 5: browser inference and splat rendering (weeks 6–7)

1. ONNX export with symbolic shapes; a parity test asserting PyTorch and onnxruntime agree to 10^{-4} over a 100-step rollout, *before* any frontend code touches the model.

2. Next.js + React Three Fiber app: load the scene packet, ONNX Runtime Web (WebGPU, WASM fallback), TypeScript graph builder and integrator, splat rendering with per-body batches, OBB picking, impulse UI, sliders, reset.

3. Reconstruction endpoint: FastAPI on Modal, or precompute scene packets and serve them as static files, which removes all server cost and cold starts while reviewers still get full interactivity.

4. Deploy to Vercel.

Deliverable: public URL. Photo-derived objects, poked in a browser, physics by neural network, client-side.

8.7 Milestone 6: polish and writeup (week 8)

Dark-mode UI pass, a 60-second screen recording, a README with the architecture diagram, evaluation table, and honest limitations. Optionally the multi-view capture path for a hero scene.

9 Risks and Fallbacks

Risk	Likelihood	Fallback
Per-body head underfits contact detail	Medium	Per-particle head + Kabsch projection (built in Milestone 2, one config switch)
Single-image recon too poor to cluster	Medium	Multi-view phone capture path; or reconstruct each object separately and compose
LGM/TripoSR broken on MPS	Medium	Run Stage A on Colab or a rented GPU; it is offline and one-shot
ONNX export op gaps	Low–Med	Implement scatter/graph ops in TypeScript, keep only MLPs in ONNX; or WebSocket + local server for live demos and a recorded video for the public page
Browser too slow at 60 Hz	Low	30 Hz physics + interpolation; coarser particle spacing
Rolling never settles (pencils creep or jitter at rest)	Medium	Overweight low-energy rolling states in the data mix; settle damping below a kinetic-energy threshold; if needed, restrict the public demo to non-rolling shapes
Long-rollout drift despite noise injection	Medium	Stronger noise, longer rollout fine-tuning, soft settle damping near zero kinetic energy
MPS training too slow	Low	One A100 day, $< \$30$; the code is device-agnostic

The compound risk profile is favorable because every medium risk has a fallback that is built into an earlier milestone rather than bolted on later.

10 Budget

Item	Cost	Notes
Data generation	\$0	PyBullet, CPU, local
Training	\$0 – \$25	MPS free; optional A100 day for the final model
Reconstruction	\$0	Local MPS, or free Colab for LGM
Hosting	\$0	Vercel Hobby; static scene packets; client-side inference
Test objects and a phone	already owned	
Total	**\$0 – \$25**	

References

[1] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, P. Battaglia. *Learning to Simulate Complex Physics with Graph Networks.* ICML 2020.

[2] K. Allen, T. Lopez-Guevara, Y. Rubanova, et al. *Learning Rigid Dynamics with Face Interaction Graph Networks.* ICLR 2023.

[3] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. *3D Gaussian Splatting for Real-Time Radiance Field Rendering.* SIGGRAPH 2023.

[4] J. Tang, Z. Chen, X. Chen, T. Wang, G. Zeng, Z. Liu. *LGM: Large Multi-View Gaussian Model for High-Resolution 3D Content Creation.* ECCV 2024.

[5] D. Tochilkin, D. Pankratz, Z. Liu, et al. *TripoSR: Fast 3D Object Reconstruction from a Single Image.* arXiv:2403.02151, 2024.